HW1 Design Report: A Bounded Agent for Bandit Alert Classification

Anubha Vyasamudri (av3329)
Sanjita Chandan Ballapur (sb5216)

The implementation is in src/hw1_agent/; setup and run instructions are in README.md. The results are taken from the committed traces in runs/.

Implementation at a glance

- **Language and runtime:** Python 3.12 and Bandit 1.9.4, invoked as python -m bandit from the same interpreter.
- **Model:** OpenAI gpt-5.4-mini through Chat Completions with function tools, reasoning_effort = "none" and tool_choice = "required".
- **Agent design:** one ReAct-style loop per alert, implemented as a state machine in graph.py rather than an agent framework.
- **Tools:** read_file, search_code and list_dir; submit_verdict ends an alert.
- **Budget:** prompt and completion tokens, checked before each call using a byte-based upper bound on the next input.
- **Modules:** cli and config set up graph.py, which handles nodes, budget checks, failsafes, settlement and crashes. It uses scanner, workspace, tools, dispatcher, provider, trace and report.

A1. Behaviours

For each alert the agent:

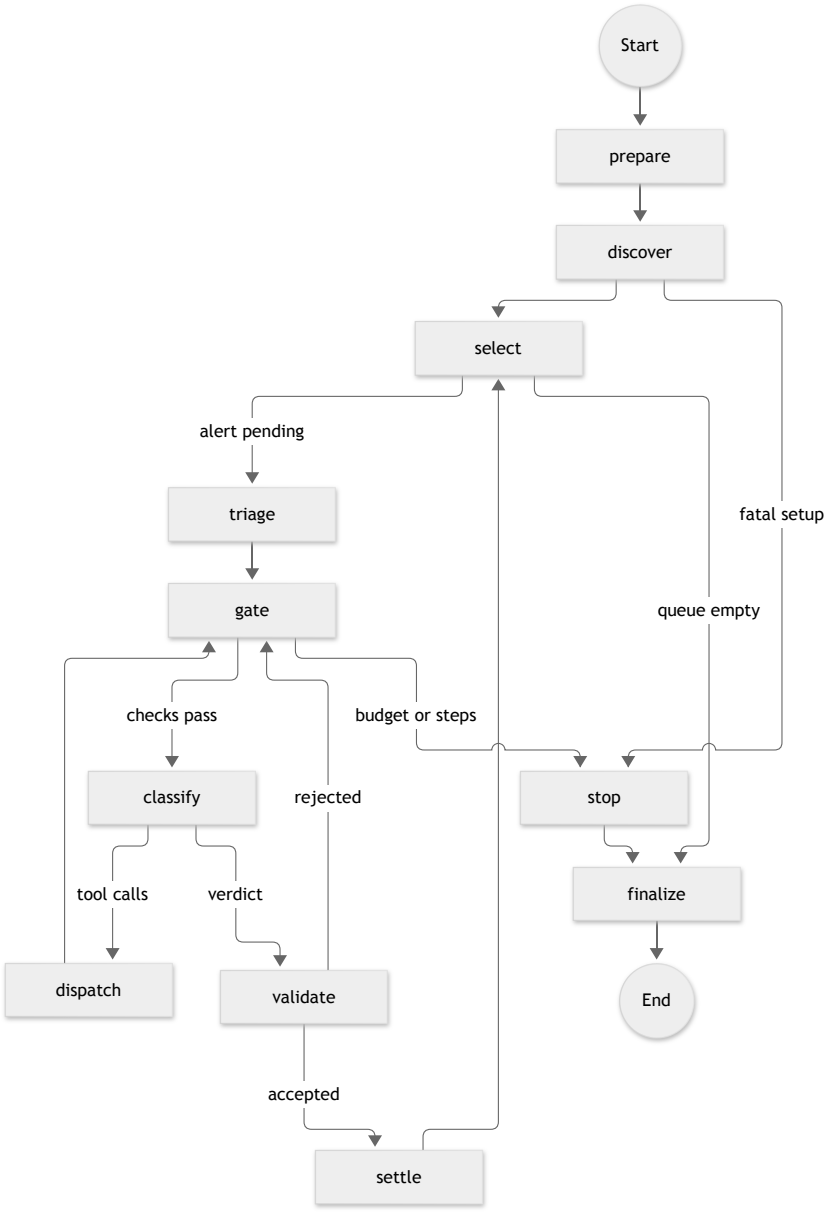
1. Hides every other OWASP BenchmarkTest* case (A7) and builds a context message: the Bandit fields (test id, CWE, severity, confidence, issue text).
2. Asks the model to trace the value that reaches the sink back to its source. Is it attacker-influenced or constant, trusted or sanitised, and on which paths? The model proposes read_file, search_code or list_dir calls, and the runtime executes them.
3. Accepts a submit_verdict only after the runtime validates it. The label must be 0 or 1, the rationale must cite at least one line, and every evidence range must lie in a file the model may read and inside lines the runtime actually showed the model for this alert.
4. Settles the alert's report row exactly once (classified, error or budget_exhausted) and moves on to the next alert.

The agent must never:

- **Run target code.** Nothing from the target is imported, installed or executed. Bandit is a static analyser, invoked as python -m bandit with an argument list (no shell). The tools only read bytes.
- **Write to the target.** All tool access goes to a disposable copy. Bandit writes its JSON to a temp directory outside the target. There are no write tools.

- **Invent observations.** Only the dispatcher writes observations (tool messages). The model's text is never treated as an observation. Evidence must be a subset of lines the runtime displayed, so a verdict that cites code the model never saw is rejected.
- **Read withheld material**: .git/, expectedresults*.csv, or another OWASP case (A7).
- **Invent usage.** The budget charges only provider-reported counters. A call that fails before usage is returned records no usage and no model_call event.

A2. State machine



The figure shows the main alert loop and primary stop paths; A3 lists every conditional edge.

Any uncaught exception, at any node, goes to the crash handler (Agent._crash). It settles every unsettled alert as error, writes report.json, and writes the single terminal event (fatal_error).

A3. Nodes and edges

Node	Does	Outgoing edge(s)	Kind	Decided by
prepare	resolve commit, open the workspace, build the provider	then discover	fixed	N/A
discover	Bandit tool_request/tool_result, normalise, sort, assign ids	then select / stop	conditional	runtime
select	pick the next unsettled alert	then triage / finalize	conditional	runtime (routing triage / finalize)
triage	per-case withholding, context message, state_change	then gate	fixed	N/A
gate	turn limit, then max_steps, then token budget	then classify / settle / stop	conditional	runtime (turn_limit / max_steps / budget_exhausted)
classify	one model call with tool_choice="required" (forced submit_verdict on the last turn), model_call event with reported usage	then dispatch / validate	conditional	model (routing tools / submit)
		then gate (nudge) / settle (error) / stop (fatal)	conditional	runtime (no_tool_call / error / fatal_error)
dispatch	run each requested tool through the dispatcher	then gate	fixed (default)	N/A
		then stop	conditional	runtime (no_progress / max_steps)
validate	check the verdict	then settle / gate / stop	conditional	runtime (settle / reject_verdict / no_progress)
settle	write the row, one state_change with alert_status + label, restore withheld case files	then select	fixed	N/A
stop	settle every remaining alert (budget_exhausted, or error for the other stops)	then finalize	fixed	N/A
finalize	write report.json, then the one terminal event	end	fixed	N/A

Every conditional edge taken writes exactly one routing event with by. Fixed and default edges are not traced. The model chooses only between investigating (tools) and finishing (submit). Every other transition is the runtime's. The no_tool_call nudge edge is a fallback: since tool_choice became required, it fired 0 times across the 200 model calls of the six runs.

A4. Agent state

Field	Reducer	Lifetime	Writer	Model sees?
alerts (report rows: status, label, rationale, evidence)	set once per alert in _settle (a second write raises)	run	runtime (settle, stop, crash handler)	no
queue (unsettled alerts)	remove on settle	run	runtime	no
usage (input, output)	add, from provider-reported counters	run	runtime (classify)	no
trace usage mirror (incl. cache_read)	add, from written model_call events only	run	Trace.event	no
steps_used	add 1 per model call and per tool execution	run	runtime (classify, dispatch)	no
no_progress_streak	reset to 0 on progress or settlement, else +1 per turn	run	runtime	no
provider_failures	+1 per failed call, reset on success	run	runtime	no
first_call_input	set once (first reported first-turn input)	run	runtime	no
stop (status, reason)	set once	run	runtime	no
session.messages	append	alert	runtime (system/context/nudges/tool results) + model (its own assistant turns)	**yes**
session.observed (path then line numbers shown)	union	alert	dispatcher / triage	no
session.seen_requests (tool+args fingerprints)	union	alert	dispatcher	no
session.turns	add	alert	runtime	no (except the final-turn instruction)
session.last_input, last_bytes	overwrite	alert	runtime	no
session.pending_reply	overwrite	one turn	runtime (classify)	it is the model's own reply
workspace active_case, stash	set per alert, restored at settle	alert	workspace	no

The model never writes state directly. Its only outputs are proposals, tool calls and verdicts, and the runtime validates each one before anything changes.

A5. Tools

All tools take repo-relative POSIX paths and go through Workspace.resolve (A7) before touching disk. A refusal comes back as an observation beginning REFUSED: with a tool_result ok=false error=. . . event. The dispatcher refuses an unknown tool name, arguments that are not a JSON object, arguments that fail the tool's JSON Schema (wrong type, missing or extra keys), and an exact repeat of an earlier request for the same alert. Results are capped at 12,000 characters. On truncation, only the lines still visible count as observed.

Tool	Arguments	Returns	Limits	Refusals
read_file	path, start_line ≥ 1, end_line ≥ start_line	numbered lines + [file has N lines; showing a-b]	≤ 200 lines per call; lines > 400 chars truncated	path refused (absolute, .., outside the copy, withheld); not a file; binary; start_line past EOF; end_line < start_line
search_code	pattern (literal, 1-200 chars), path (default .), glob (file-name pattern, default *.py), case_sensitive	path:line: text hits	≤ 50 hits; files > 2 MB skipped; ≤ 30,000 files scanned; literal match only (no regex, so no ReDoS)	bad pattern/glob; path refused or missing. Withheld files are silently skipped.
list_dir	path	sorted entries, dirs end in /	≤ 200 entries	path refused; not a directory. Withheld entries are not listed.
submit_verdict (finish)	label ∈ {0,1}, rationale, evidence[1..5] of {path,start_line,end_line}	Ends the alert	ranges ≤ 200 lines; rationale stored up to 3,000 chars	rejected if: called together with other tools; schema fails; rationale empty or cites no line; evidence path refused or not a file; any cited line was never shown to the model

Bandit is the runtime's discovery tool, traced as tool_request/tool_result with tool: "bandit". The model cannot call it.

A6. Start, stop and termination

Start. The CLI parses --input/--output/--config. The config must parse as TOML and validate against config.schema.json. If it does not, the run never starts and the trace ends config_invalid. [agent] keys (max_turns_per_alert, no_progress_limit, request_timeout_s) are optional.

Terminal statuses. Closed and mutually exclusive. Exactly one is written, as the last event.

Status	Trigger	Rows still unsettled become
completed	every alert settled by the agent loop	N/A
budget_exhausted	budget gate: required > max_tokens − used before a model call	budget_exhausted, label: null
max_steps	steps_used + 1 > max_steps before a model call or a tool execution	error ("run stopped (max_steps)")
no_progress	no_progress_limit consecutive model turns with no new observation: no tool call, all requests refused or duplicated, or verdict rejected	error
fatal_error	uncaught exception, Bandit failure, unusable provider (missing key, auth, unknown model, invalid parameter, quota), or 3 consecutive failed calls	error
config_invalid	config rejected before any scan or model call	(no rows)

Precedence is fixed by where each check runs. A crash pre-empts everything. In gate, max_steps is checked before

the budget. no_progress is evaluated after a turn's observations. The first check to fire stops the run, so two statuses can never both apply.

Budget policy.

- *Charge*: used = Σ (input + output) over model_call events, as reported by the provider. For OpenAI, input = prompt_tokens and output = completion_tokens, which includes reasoning tokens. cache_read = prompt_tokens_details.cached_tokens is recorded and never charged. OpenAI reports no cache-write counter. report.budget.used_tokens is computed from the trace's own mirror of the model_call events, so it equals the trace sum by construction.
- *Output allowance*: budget.output_allowance is sent as max_completion_tokens on every call.
- *When it is checked*: in gate, before every model call. Required = projected_input + output_allowance, and the call is made only if required ≤ max_tokens − used.
- *Projection* (gate only, never charged): an upper bound built from provider numbers. A BPE token covers at least one byte, so the UTF-8 bytes added since the previous call bound the tokens added: projected = last reported input + bytes added + 16/message. A new alert's first call uses the first reported first-turn input plus the new alert message's bytes. Before any report exists, it uses the whole request's bytes. Because the projection is an upper bound and output is capped by the allowance, used ≤ max_tokens always holds.
- *At the cutoff*: a routing event budget_exhausted by runtime records remaining and required. The current and all later alerts get budget_exhausted rows (one state_change each). Then report.json is written with exhausted: true, followed by terminal budget_exhausted.

Per-alert bound. At most max_turns_per_alert (8) model calls per alert. Every call is sent with tool_choice = "required", so each turn is either an investigation or a verdict. On the last one the runtime forces tool_choice = submit_verdict. If that verdict is still rejected, the alert settles error and the run continues. In the real runs this forced turn was used once (radicale/storage/multifilesystem/lock.py:102, B602: 7 investigation calls, then the forced verdict).

A7. Workspace and withholding

Withheld	Targets	Why	How	Restored
.git/	all	history may contain commits after the pin, including fixes	never copied (copytree(ignore=.git)); resolve refuses any .git component	nothing moved; the original is untouched
expectedresults*.csv	OWASP (rule applies everywhere)	the answer key	moved from the copy into a sibling stash (repo-withheld/) for the whole run; resolve refuses the name	moved back on workspace exit, then the copy is deleted
every BenchmarkTest* file except the current case's files (.py and its `templates/web/*/BenchmarkTest<id>.html`)	OWASP	each case has a safe twin, so the answer is one grep away	moved to the stash in triage for that alert; resolve refuses non-active case names	moved back in settle, and on exit

One path check. Workspace.resolve(path) is the only route from a tool argument to the file system. It refuses non-strings, NUL bytes, absolute paths (including drive letters), any .. component, withheld names, and anything whose fully resolved path, with symlinks followed, lies outside the copy, or whose resolved target is withheld.

search_code and list_dir call resolve on every file or entry they would return, so withheld files are invisible to them too, not only to direct reads. Discovery (Bandit) scans the original --input in full, as required, and never writes to it. helpers/ is not withheld.

Implementation path and changes

We first implemented and tested the agent offline, then ran it with the model API. The changes described below followed from failures observed during development and from the corresponding traces.

1. Offline build. We implemented and checked the components in this order: config validation, then the Bandit wrapper (reproducing the brief's counts of 10 / 21 / 18 at the pinned commits), then the workspace and path check, the trace and report writers, and finally the graph. To exercise the graph without spending tokens we drove it with a scripted model and with a local stand-in for the OpenAI endpoint, and checked every run against the three schemas and the six trace invariants. The development harnesses are not included in the submission.

2. Provider: OpenAI, and a budget gate without a tokenizer. We use OpenAI because that is the API key we had. Chat Completions has no free endpoint for counting input tokens, and tiktoken downloads its encoding files at first use, which adds a network dependency. We kept the gate dependency-free instead: every BPE token covers at least one byte, so the UTF-8 bytes added to the conversation since the last provider-reported input are a hard upper bound on the tokens added (A6). The bound only gates calls. What is charged is always the provider's own prompt_tokens + completion_tokens.

3. Bandit version check. python -m bandit --version prints __main__.py 1.9.4, not bandit 1.9.4, so our first version check failed every scan. We now parse the version from the first line, whatever the program name, and still refuse anything other than the configured 1.9.4.

4. First real run: fatal_error with 0 steps. The provider could not be built because OPENAI_API_KEY was not set in that shell. Bandit still ran, all 10 Radicale alerts got error rows, and the trace ended with one terminal fatal_error. This was an unplanned but real check of the fatal path.

5. reasoning_effort = "low" was rejected. The first call returned HTTP 400: *"Function tools with reasoning_effort are not supported for gpt-5.4-mini in /v1/chat/completions . . . set reasoning_effort to 'none' "*. Because provider.py classifies a BadRequestError as fatal (it would repeat on every call), the run stopped after 1 step instead of failing all 10 alerts one by one. We set reasoning_effort = "none" in both configs. We did not move to the Responses API, because that would have meant rewriting the provider for a model setting.

6. no_progress on the first full OWASP run then tool_choice = "required". With reasoning off, the model often answered in prose instead of calling a tool. On BenchmarkTest00012 it needed one nudge. On BenchmarkTest00013 it read the case and helpers/utils.py and then replied four times in a row with finish_reason = stop and no tool call. The no-progress detector fired as designed (4 consecutive turns without progress), ending the run after 2 of 18 alerts. That trace was replaced when we re-ran all six runs, and the event sequence above is taken from it. The fault was in our request rather than the detector: with tool_choice = "auto" the model is allowed to skip tools, but in this design every turn must be a tool call or a verdict. We changed it to "required". After the change, every one of the 200 model calls across the six runs ended in a tool call, and the nudge path never fired. We then re-ran all six runs so that the committed outputs come from one version of the code.

7. Budgets. configs/full.toml (1,500,000 tokens) was sized for SGLang, the largest target. It used at most 215,298. configs/low.toml (50,000) was kept because it cuts every target partway: Radicale after 3 of 10 alerts, SGLang after 3 of 21, and OWASP after 15 of 18.

C2. Results

Model: gpt-5.4-mini (reasoning_effort = "none"). Each run's config is the corresponding file in configs/targets/.

run	alerts	label 1	label 0	null	used / max tokens	terminal
radicale-full	10	6	4	0	161304 / 1500000	completed
radicale-low	10	0	3	7	37278 / 50000	budget_exhausted
sglang-full	21	13	8	0	215298 / 1500000	completed
sglang-low	21	3	0	18	40941 / 50000	budget_exhausted
owasp-full	18	10	8	0	47336 / 1500000	completed
owasp-low	18	6	9	3	44075 / 50000	budget_exhausted

OWASP precision / recall (full run): TP=6, FP=4, FN=2, TN=6 then precision 0.600, recall 0.750.

How the labels were used to refine the agent. On the full OWASP run the agent labelled 12 of the 18 alerts correctly. We compared each wrong label with its rationale and trace, and the errors fall into three groups:

- *A guard was seen but not trusted* (false positives: 00073 B307, 00075 B102). Both rationales cite the check on lines 46-50 that only lets a quoted string literal through, yet still treat eval/exec as reachable from the cookie. The model took "the value comes from a cookie" as enough, even though the label definition says a value neutralised on every path is a 0.
- *Parser defaults were assumed unsafe* (false positives: 00017 B317 and B318). The model reasoned that a cookie-controlled document reaching xml.sax and minidom "without defusing" means XXE, without checking whether those parsers resolve external entities by default.
- *Weak hashes were judged by how the digest is used* (false negatives: 00058, 00060, B324). The model called SHA-1 safe because the digest is only written to a file and gates no security decision. In 00060 it also leaned on the test/dev-only exemption because the file sits under testcode/, which is true of every OWASP case. The benchmark marks both as vulnerable.

We did not change the prompt in response to these errors. The safe-use criteria and the "check every path" instruction were already in the system prompt from the first run, and the errors above happened anyway. They show that the model applied those rules unevenly rather than that the rules were missing. The labels shaped our conclusions about where the next changes should go, and we deliberately did not apply them to these 18 cases, because that would only tune the agent to the development set while Radicale and SGLang are held out:

- ask the model to confirm that a guard can actually be bypassed before labelling a guarded sink 1;
- ask it to check a parser's defaults instead of assuming them;
- apply the test/dev-only exemption only when the code really cannot run in production, never because of a directory name;
- state that a weak hash on a reachable path is a finding regardless of what the digest is used for.

The changes we did make during development were to the runtime and the request, not to the labelling rules.

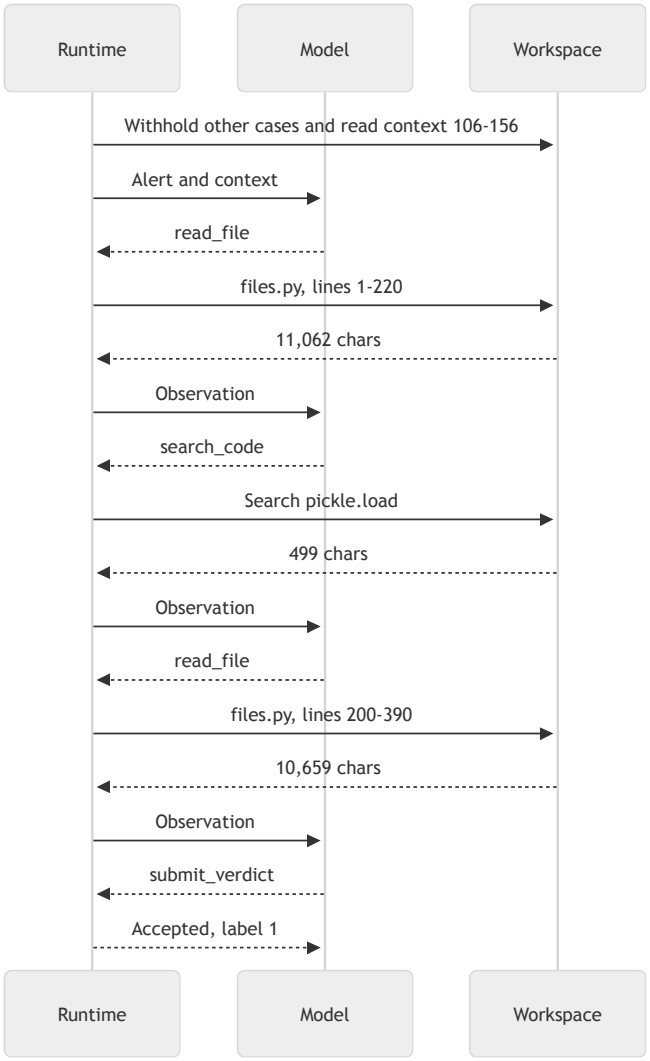
Other measured behaviour.

run	model calls	calls per classified alert (mean / max)	tokens per classified alert (mean / max)	cache_read share of input	wall time
radicale-full	46	4.6 / 8	16,130 / 37,867	65%	45 s
sglang-full	73	3.5 / 7	10,252 / 29,012	45%	83 s

run	model calls	calls per classified alert (mean / max)	tokens per classified alert (mean / max)	cache_read share of input	wall time
owasp-full	28	1.6 / 3	2,630 / 5,150	34%	51 s

- *Investigation depth follows the target.* On OWASP, 10 of 18 alerts were decided from the ±25-line triage context alone, with no tool call, because each case is self-contained. Radicale and SGLang needed cross-file reads: 5 of their 69 evidence ranges cite a file other than the flagged one.
- *The runtime checks fired on real model output.* In owasp-low, the first verdict for BenchmarkTest00059 was rejected because its rationale cited no line. The model resubmitted on the next turn and the verdict was accepted. In sglang-full, a read_file on a directory path came back as a refusal, and the model continued. The forced final turn was used once (A6).
- *The budget gate is safe but conservative.* used_tokens ≤ max_tokens held in every run. At the Radicale low cutoff, 12,722 tokens remained but the next call required 13,624: a projected input of 9,528 plus the 4,096 output allowance. The largest output of any call across all six runs was 247 tokens (mean 85), so most of the reserved headroom is the output allowance. An allowance of about 1,024 would let the same budget cover more alerts.
- *Charged input includes cached input.* OpenAI cached 34-65% of the prompt tokens on the full runs (cache_read). We record it and do not charge it separately, but prompt_tokens already includes it, so our charge is the conservative one.
- *Labels are not stable across runs.* radicale/sharing/files.py:131 (B301) was labelled 1 in radicale-full and 0 in radicale-low. Within one run, similar pickle-cache sinks also diverged: cache.py:109 was labelled 1, while history.py:53 and sync.py:79 were labelled 0. We do not fix a sampling seed, and one run per configuration cannot separate prompt effects from sampling noise.
- *Low-budget OWASP run:* 15 of 18 alerts were classified before the cutoff, giving precision 0.50 and recall 0.50 (TP 3, FP 3, FN 3, TN 6, 3 unlabelled).

Trajectory of one alert (from runs/radicale-full/trace.jsonl):



Trace details. Alert radicale-radicale-sharing-files.py-131-B301 used model calls at steps 36, 40, 44, and 48, with reported input/output tokens 1,621/32, 4,446/35, 4,638/32, and 7,222/174. The runtime routed triage, three tool turns, submit, and settle.

C3. Limitations and alternatives

Limitations of this design.

- One ReAct-style loop per alert, so alerts run strictly one after another. Latency is the sum of the per-alert loops: 45-83 s per full run here.
- The byte-based budget projection plus a 4,096-token output allowance is safe but pessimistic. At the Radicale low cutoff it refused a call with 12,722 tokens left, while no call ever produced more than 247 output tokens.
- no_progress and max_steps end the whole run, as the brief requires, even when only one alert is stuck. We saw this for real: one alert answering in prose ended the first full OWASP run after 2 of 18 alerts. tool_choice="required" and the per-alert turn cap with a forced final submit now keep one stuck alert from triggering them in ordinary cases.
- Labels are not reproducible from run to run (C2), and with a single run per configuration the measured

precision and recall carry sampling noise.
- max_steps (600) never fired on a real run. The largest run used 126 steps, so this failsafe is shown by its implementation (the check in gate and dispatch) rather than by a committed real trace.
- The evidence check proves the model *saw* the cited lines, not that its reasoning about them is right.
- The context is rebuilt from scratch for each alert. Nothing learned about shared helpers carries over.

Alternatives:

This agent: ReAct loop per alert. The model chooses tools until it submits a verdict, while the runtime checks each step. The measured cost is 1.6 calls per OWASP alert to 4.6 per Radicale alert, and grows with the transcript. Turns run sequentially within an alert. There is one actor, so the runtime can keep state and settlement rules in one place. Bad requests become refusals, while duplicate requests, a turn cap and a no-progress check limit loops. This design fits alerts that need a variable amount of cross-file investigation.

Fixed workflow. The runtime follows the same steps for every alert: provide context and make one classification call. This has the lowest and most predictable cost, and alerts can be processed in parallel. It works well for single-file OWASP cases but is less suitable for Radicale or SGLang, where relevant context may be spread across files.

Plan-and-execute. A planner would write an investigation plan for each alert and an executor would carry it out, with optional re-planning. This may help with large repositories such as SGLang, but planning adds calls and latency. A poor plan could spend the budget on irrelevant reads, and re-planning introduces another possible loop.

Supervisor / worker. A supervisor could route alerts or sub-questions to specialised workers, such as a taint tracer or crypto-usage checker, and aggregate their results. Workers could run in parallel, but coordination and aggregation would add overhead. Shared state, conflicting verdicts and worker failures would make it harder to preserve the one-settlement-per-row rule. For runs of 10-21 alerts, this seems unnecessarily complex.